



SimSpect

Automated litmus testing for simulated hardware
Spectre defenses

Anna Eaton, 2nd year Ph.D. student

Hardware side-channel attacks (HSCAs): Attackers exploit data-dependent hardware resource usage to infer operand value, under some execution circumstance.

Architecturally determined execution path passes unsafe instruction

Constant-time code (SW)

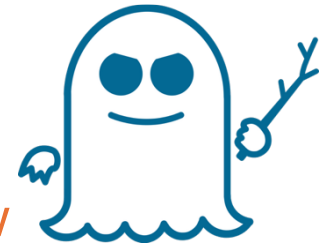
Transient, microarchitecturally determined execution path passes data to unsafe instruction



Meltdown attacks exploit privileged data permissions exception handling

Kernel page table isolation (KPTI)

Spectre attacks use architecturally legal instructions + incorrect speculative control flow



STT

- All memory
- All registers
- Architecturally untransmitted memory
- Annotated pages and dependent registers
- ...

×

- Load leaks address
- Branch leaks branch condition
- ALUop leaks operand
- Store leaks address
- ...

×

- Store-to-load forwarding address prediction
- Branch address prediction
- Branch target prediction
- Load value prediction
- ...

Hardware side-channel attacks (HSCAs): Attackers exploit data-dependent hardware resource usage to infer operand value, under some execution circumstance.

Architecturally determined execution path passes unsafe instruction

Constant-time code (SW)

Transient, microarchitecturally determined execution path passes data to unsafe instruction



Meltdown attacks exploit privileged data permissions exception handling

Kernel page table isolation (KPTI)

Spectre attacks use architecturally legal instructions + incorrect speculative control flow



- All memory
- All registers
- Architecturally untransmitted memory
- Annotated pages and dependent registers
- ...

×

- Load leaks address
- Branch leaks branch condition
- ALUOp leaks operand
- Store leaks address
- ...

×

- Store-to-load forwarding address prediction
- Branch address prediction
- Branch target prediction
- Load value prediction
- ...

Hardware side-channel attacks (HSCAs): Attackers exploit data-dependent hardware resource usage to infer operand value, under some execution circumstance.

Architecturally determined execution path passes unsafe instruction

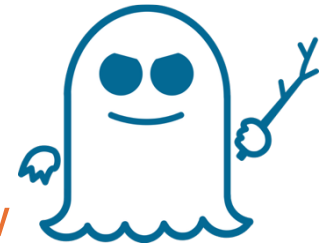
Constant-time code (SW)

Transient, microarchitecturally determined execution path passes data to unsafe instruction

Meltdown attacks exploit privileged data permissions exception handling

Kernel page table isolation (KPTI)

Spectre attacks use architecturally legal instructions + incorrect speculative control flow



- All memory
- All registers
- Architecturally untransmitted memory
- Annotated pages and dependent registers
- ...

×

- Load leaks address
- Branch leaks branch condition
- ALUop leaks operand
- Store leaks address
- ...

×

- Store-to-load forwarding address prediction
- Branch address prediction
- Branch target prediction
- Load value prediction
- ...

Hardware side-channel attacks (HSCAs): Attackers exploit data-dependent hardware resource usage to infer operand value, under some execution circumstance.

Architecturally determined execution path passes unsafe instruction

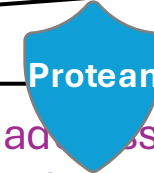
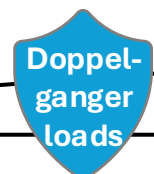
Constant-time code (SW)

Transient, microarchitecturally determined execution path passes data to unsafe instruction

Meltdown attacks exploit privileged data permissions exception handling

Kernel page table isolation (KPTI)

Spectre attacks use architecturally legal instructions + incorrect speculative control flow



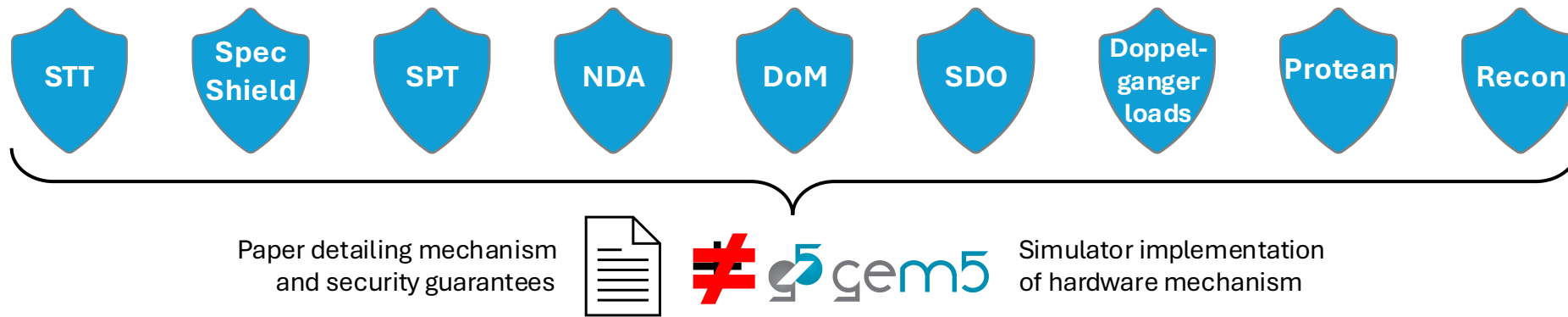
- All memory
- All registers
- Architecturally untransmitted memory
- Annotated pages and dependent registers
- ...



- Load leaks address
- Branch leaks branch condition
- ALUOp leaks operand
- Store leaks address
- ...



- Store-to-load forwarding address prediction
- Branch address prediction
- Branch target prediction
- Load value prediction
- ...



- **Many HW-modifying Spectre defenses** — with different mechanisms and security guarantees
- **PPA essential** in HW modifications, simulator performance **invalid** if it doesn't uphold security guarantees
 - Frequent bugs in security contract of implementation, need faster iteration

Given an **arbitrary proposed hardware Spectre defense**, with a gem5 architectural simulator proof of concept, can we

1. **formalize the security properties** that the defense will uphold
2. **rigorously validate** them on the implementation?

We want a **comprehensive** set of tests that covers the space of leakage that a particular defense should protect against:

1. Defense-specific
2. Targeted to cause leakage

Prior work

SPEC2017
Benchmark suite

- Tests to an incorrect security contract
- Tests do not target leakage

Spectre benchmark
suite

- Tests to an incorrect security contract
- Tests target leakage

Handmade
defense- specific
litmus tests

- Tests are not comprehensive
- Tests target leakage

AMuLeT (security
contract targeted
fuzzing)

- Tests to an overly broad security contract
- Tests do not target leakage (random)

SimSpect:

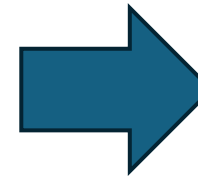
formal language

+

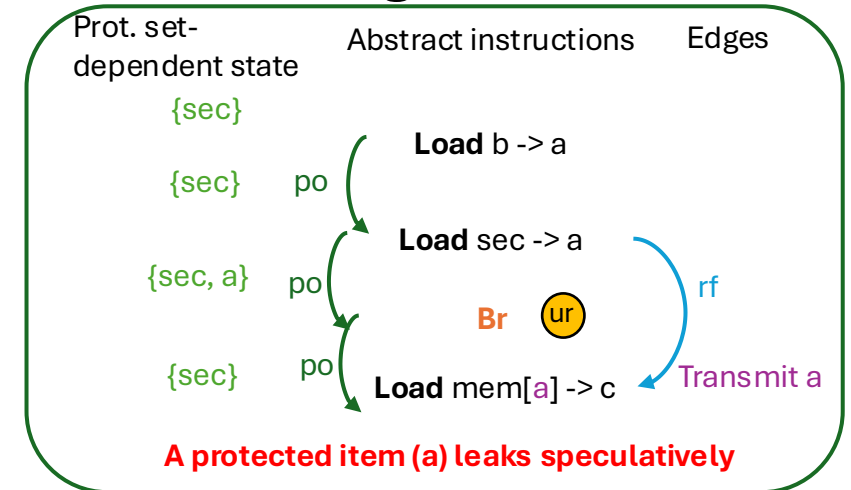
litmus test
generation/execution
pipeline

Each defense has:

1. **Protection set** - the set of architectural state that the defense promises to not leak
2. **Leakage contract** - the set of pairs of transmitters and their unsafe operand(s)
3. **Execution contract** - what speculation primitives the defense considers



Violating execution



Expand into assembly

Run on gem5

Check if the violation occurs

- All memory
- All registers
- Architecturally untransmitted memory
- Annotated pages and dependent registers
- ...

×

- Load leaks at
- Branch leaks condition
- ALUop leaks
- Store leaks a
- ...

Hardware side-channel attacks (HSA) exploit data-dependent hardware re: operand value, under some executio

```
Alloy Analyzer 6.0.0 built 2021-10-20T12:40:15.600Z

File Edit Execute Options Window Help
New Open Reload Save Execute Show

}
else Elected' = Elected
}

fact {
  // initially inbox and outbox are empty
  no inbox and no outbox
  // initially there are no elected nodes
  no Elected
}

fact traces {
  // possible events
  always {
    stutter or
    some n : Node | initiate[n] or
    some n : Node, i : Id | send[n,i] or
    some n : Node, i : Id | process[n,i]
  }
}

run example {
  eventually some Elected
} for exactly 3 Node, exactly 3 Id

run two_process_in_a_row {
  eventually (
    (some n : Node, i : Id | process[n, i]) and
    after (some n : Node, i : Id | process[n, i])
  )
} for exactly 3 Node, exactly 3 Id

Line 96, Column 1 [modified]
```

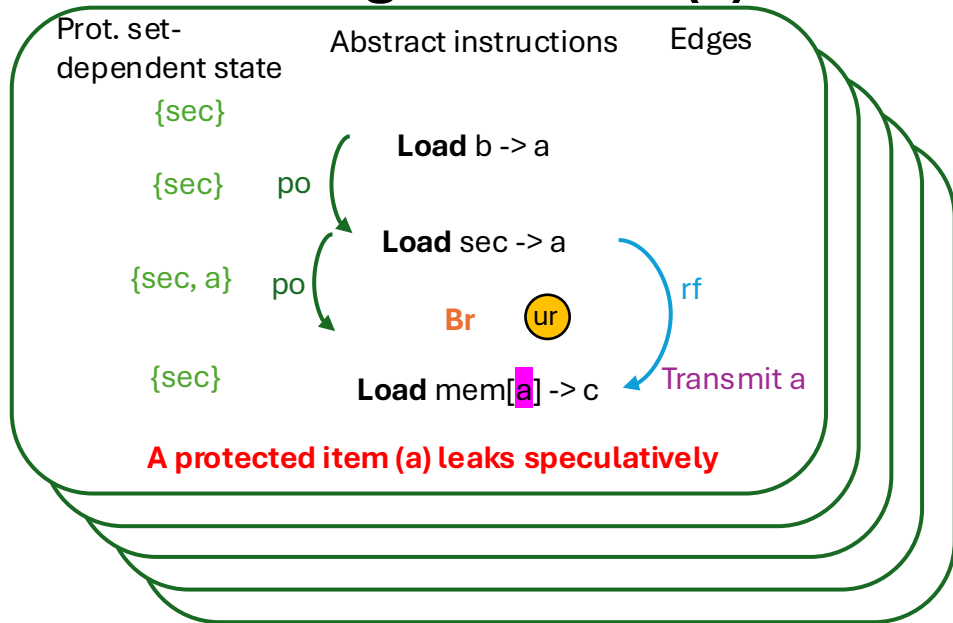
```
Alloy Analyzer 6.0.0 built 2021-10-20T12:40:15.600Z

Executing "Run example for exactly 3 Node, exactly 3 Id"
Sig this/Node scope <= 3
Sig this/Id scope <= 3
Sig ordering/Ord scope <= 1
Solver=minisat(jni) Steps=1..10 Bitwidth=4 MaxSeq=4
1..8 steps. 21652 vars. 972 primary vars. 40833 cla
Instance found. Predicate is consistent. 35ms.

Executing "Run example for exactly 3 Node, exactly 3 Id"
Sig this/Node scope <= 3
Sig this/Id scope <= 3
Sig ordering/Ord scope <= 1
Solver=minisat(jni) Steps=1..10 Bitwidth=4 MaxSeq=4
1..8 steps. 21652 vars. 972 primary vars. 40833 cla
Instance found. Predicate is consistent. 29ms.
```

litmus test generation/execution pipeline

Violating execution (s)



Concretization process

Pass 1: Specify instructions in LLVM, annotate speculation state

Pass 2: Add interleaved instructions on- and off-path of leakage

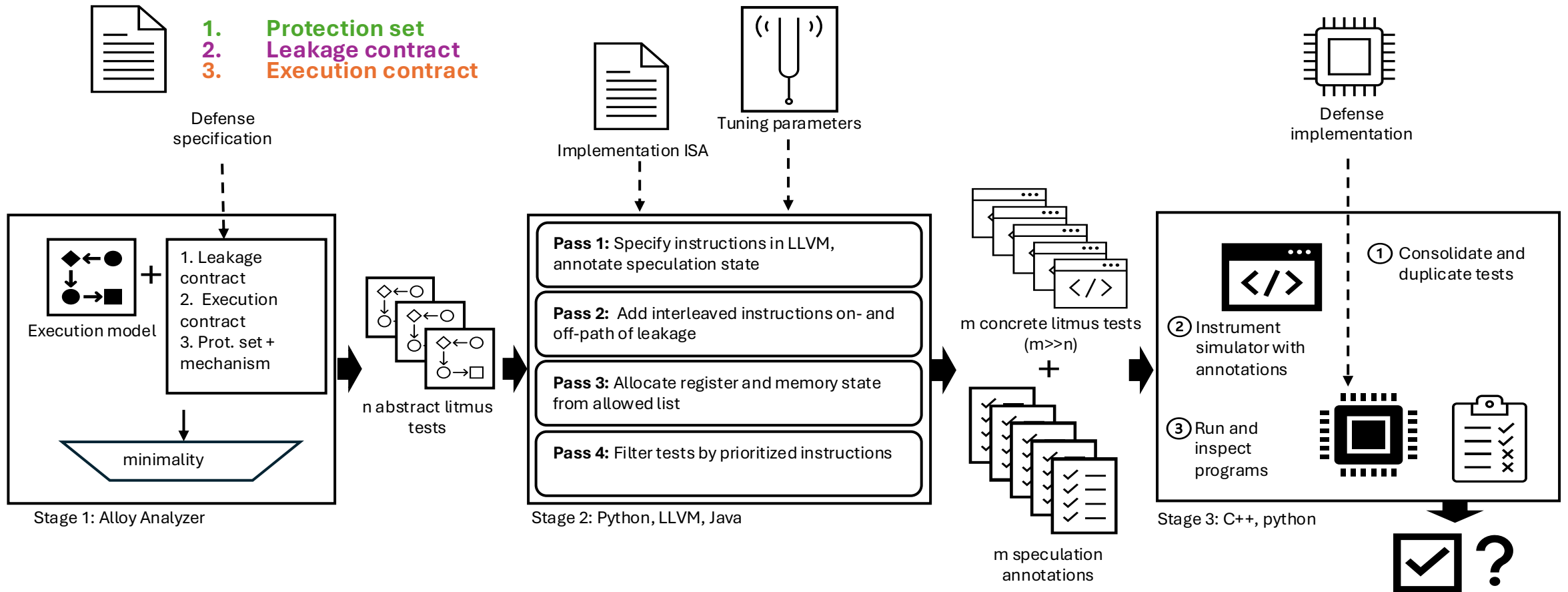
Pass 3: Allocate register and memory state from allowed list

Pass 4: Filter tests by prioritized instructions

```
mov r1 [r2]
mov r3 [r1] // leak r2, unprotected
mov r1 [r4]
jmp // mispredicted not taken
mov r5 [r1]
```

Annotation goes into
predictors in gem5

SimSpect zoomed-out pipeline:



QUESTIONS?